

CSE231: Programming Assignment 1

SimpleLoader — A From-Scratch ELF Loader & x86 Bootloader

Contents

- 1. Introduction
- 2. Part A — SimpleLoader (objective, ELF structures, algorithm, mmap configuration, error handling, directory structure, without-bonus vs with-bonus)
- 3. Design Decisions & Challenges
- 4. Part B — x86 Bootloader (implementation, build & run, conceptual questions)
- 5. Testing & Verification (evidence of working builds)
- 6. Key Learnings
- 7. Repository
- 8. Conclusion

1. Introduction

This document describes the design and implementation of Programming Assignment 1, which consists of two independent parts: (A) a SimpleLoader that manually loads and executes a 32-bit ELF executable without using any ELF-parsing library, and (B) a minimal 16-bit x86 bootloader that runs directly on BIOS via QEMU. Together, the two parts span both ends of program execution on a PC — from the very first instruction the CPU runs after power-on (Part B), to how a userspace program is loaded and handed control (Part A) — giving a concrete, hands-on view of what higher-level abstractions like "running a program" actually involve underneath. The full source code, build files, and this document are available in the GitHub repository linked in Section 7.

2. Part A — SimpleLoader

2.1 Objective

Implement a loader in C that, given a statically linked 32-bit ELF executable, parses the ELF and Program Header structures directly (using only the struct definitions from `<elf.h>`, not a parsing library), locates the loadable segment containing the program's entry point, maps that segment into memory, and transfers control to it — replicating, at a minimal level, what the OS loader does when a program is exec'd.

2.2 Relevant ELF Structures

Two structures from `<elf.h>` drive the implementation:

Elf32_Ehdr (ELF header) — fields used

Field	Purpose in this implementation
<code>e_entry</code>	Virtual address of the program's entry point
<code>e_phoff</code>	File offset where the Program Header Table

Field	Purpose in this implementation
	begins
e_phnum	Number of entries in the Program Header Table
e_ident	Magic bytes / class, used to validate the file is a 32-bit ELF

Elf32_Phdr (Program header) — fields used

Field	Purpose in this implementation
p_type	Identifies PT_LOAD segments (the ones that must be mapped into memory)
p_offset	File offset of the segment's data
p_vaddr	Virtual address the segment is linked to load at
p_memsz	Size (bytes) to allocate for the segment in memory

Per the assignment specification, p_filesz, p_paddr, p_flags, and p_align were not used — the segment copy uses p_memsz for both the allocation size and the copy length.

2.3 Loading Algorithm

load_and_run_elf() in loader.c performs the following sequence:

- 1. Open the ELF file (open) and determine its size (fstat).
- 2. Allocate a heap buffer (malloc) and read the entire file into it (read, looped until complete).
- 3. Validate the file: check the ELF magic bytes against ELF_MAGIC/SELF_MAGIC and confirm e_ident[EI_CLASS] == ELFCLASS32.
- 4. Locate the Program Header Table at (buffer + e_phoff) and iterate e_phnum entries.
- 5. Find the entry whose type is PT_LOAD and whose virtual address range [p_vaddr, p_vaddr + p_memsz) contains e_entry.
- 6. Reserve memory for that segment with mmap, and copy p_memsz bytes from (buffer + p_offset) into it.
- 7. Compute the entry point's address inside the mapped region as (e_entry - p_vaddr), added to the mmap base.
- 8. Cast that address to a function pointer of type int(*) (void) and call it — this invokes the target binary's _start.
- 9. Print the value _start() returns.

loader_cleanup() releases all resources acquired above: munmap on the mapped segment, free on the file buffer, and close on the file descriptor.

2.4 mmap Configuration

The segment is mapped using:

```
mmap(NULL, p_memsz, PROT_READ|PROT_WRITE|PROT_EXEC,
MAP_ANONYMOUS|MAP_PRIVATE, 0, 0);
```

PROT_EXEC is required because the mapped region contains executable code that will be jumped into directly. MAP_ANONYMOUS|MAP_PRIVATE requests a fresh, zero-initialized region not backed by the ELF file descriptor — the segment's contents are copied in manually with memcpy after mapping, rather than mapping the file itself.

2.5 Why This Only Works for Static, Non-PIE Binaries

The test executable (factorial.c) is compiled with -m32 -no-pie -nostdlib. Without dynamic linking or position-independent code, the addresses recorded in the ELF (p_vaddr, e_entry) can be treated as fixed offsets within the loaded segment, and no relocation or symbol resolution is needed before execution. A dynamically linked or PIE executable would require significantly more loader logic (relocations, a dynamic linker, shared library resolution) that is out of scope for this assignment.

2.6 Error Handling

Each system/library call that can fail is checked, and the loader exits with a diagnostic message rather than continuing on invalid state:

- open() returning a negative file descriptor (file missing / unreadable)
- fstat() failure
- malloc() returning NULL
- Incomplete read() (looped until the full file size is read, or an error/EOF is detected)
- ELF magic / 32-bit class mismatch (rejects non-ELF or 64-bit input)
- No PT_LOAD segment found containing the entry point
- mmap() returning MAP_FAILED

2.7 Directory Structure & Build

without-bonus (reference implementation, no bonus marks)

```
without-bonus/  
├── loader.c      (main() + load_and_run_elf() + loader_cleanup())  
├── loader.h      (unmodified, provided)  
├── factorial.c   (unmodified, provided test case)  
└── Makefile      (unmodified, provided)
```

Build & run:

```
make  
./loader factorial
```

Output: User _start return value = 120

with-bonus (submitted implementation)

The loader logic is identical to without-bonus, but restructured so load_and_run_elf() and loader_cleanup() are compiled into a shared library, with main() moved into a separate launcher that links against it at runtime.

```
with-bonus/  
├── loader/      loader.c, loader.h -> lib_simpleloader.so  
├── launcher/    launch.c           -> launch (links against the .so)  
└── test/        factorial.c         -> factorial
```

```
└─ bin/          launch + lib_simpleloader.so (populated by `make`)
└─ Makefile      top-level: builds all three, copies outputs into bin/
```

Build & run:

```
make
./bin/launch ./test/factorial
```

Output: User _start return value = 120

launch.c performs argument-count validation, then calls `load_and_run_elf(argv)` (passing the full argv so that `argv[1]`, the ELF path, is available inside the shared library), followed by `loader_cleanup()`.

2.8 without-bonus vs with-bonus — Comparison

Aspect	without-bonus	with-bonus
Entry point	<code>main()</code> inside <code>loader.c</code> itself	<code>main()</code> moved to <code>launch.c</code>
Loader form	Compiled directly into the loader binary	Compiled as <code>lib_simpleloader.so</code>
Coupling	Single self-contained executable	Launcher dynamically linked against the loader at runtime
Bonus eligible	No	Yes (up to +1%)
Core ELF-parsing logic	Identical	Identical

3. Design Decisions & Challenges

3.1 Passing argv instead of argv[1]

`loader.h` fixes the signature `void load_and_run_elf(char** exe)` — a `char**`, not a `char*`. The placeholder starter code called this with `argv[1]` (a single string), which does not match the declared parameter type. The implementation instead passes the full argv down from `main()`, so `exe[1]` inside the function correctly resolves to the ELF path. This keeps `loader.h` completely unmodified, as required, while fixing a type mismatch that otherwise would not compile.

3.2 Global state for cross-call cleanup

`loader_cleanup()` is invoked by `main()/launch.c` as a separate call after `load_and_run_elf()` returns. Since C has no way to return multiple resource handles implicitly, the file buffer pointer, the `mmap'd` segment pointer, and its length are kept as file-scope static variables (alongside the `ehdr/phdr/fd` globals already present in the starter code), so that `loader_cleanup()` can release exactly what `load_and_run_elf()` acquired, even though the two calls do not share a stack frame.

3.3 Ignoring p_filesz

The assignment's own `Elf32_Phdr` reference figure explicitly marks `p_filesz` as "ignore this for assignment". Rather than following the more general ELF convention of copying only `p_filesz` bytes and zero-filling the remainder up to `p_memsz`, this implementation copies `p_memsz` bytes directly from the file, matching the simplified model the assignment specifies. This was caught by cross-checking the implementation against the assignment's own embedded figures rather than general ELF documentation, which would otherwise have introduced a subtle mismatch with the expected implementation.

3.4 32-bit toolchain availability

Building with -m32 requires 32-bit glibc headers and libraries (gcc-multilib / libc6-dev-i386) in addition to gcc itself. On a system without these installed, compilation fails with missing-header errors (e.g. bits/libc-header-start.h) despite gcc itself being present. This was resolved by installing gcc-multilib before proceeding.

3.5 Shared-library linking at runtime

Since launch and lib_simpleloader.so are only guaranteed to sit together inside bin/ after the top-level Makefile copies them there, the launcher is linked with an \$ORIGIN-relative rpath, so it locates the shared library relative to its own location at runtime rather than depending on a fixed install path or LD_LIBRARY_PATH being set.

4. Part B — x86 Bootloader

4.1 Objective

Implement a 16-bit x86 boot sector that BIOS loads and executes directly, without any operating system present, to print a message to the screen and halt.

4.2 Implementation

The bootloader (`complete_bootloader.asm`) runs in 16-bit real mode, assembled with the origin set to `0x7C00` to match where BIOS loads the boot sector. Execution:

- SI is pointed at the start of the message string.
- A loop uses `lodsrb` to load the next character into AL and advance SI, checks for the null terminator, and if not done, sets `AH=0x0E` and calls BIOS interrupt `0x10` (teletype output) to print the character in AL, then repeats.
- On reaching the null terminator, interrupts are disabled (`cld`), the CPU is halted (`hlt`), and a tight loop (`jmp`) guards against ever resuming, in case a non-maskable interrupt wakes the CPU.
- The file is padded with zero bytes (times `510-( $\$-\$$ ) db 0`) to bring it to exactly 510 bytes, followed by the 2-byte boot signature `0xAA55`, for a total of exactly 512 bytes.

4.3 Build & Run

```
nasm -f bin complete_bootloader.asm -o boot.bin  
qemu-system-x86_64 -drive format=raw,file=boot.bin
```

QEMU displays: Hello from my bootloader!

4.4 Conceptual Questions

Why is the bootloader limited to 512 bytes?

By long-standing PC BIOS convention, a disk sector is 512 bytes, and BIOS's boot process reads exactly one sector (the first sector of the boot device) into memory and transfers control to it. Anything the bootloader needs beyond that first sector must be loaded by code within that first 512 bytes itself (e.g. reading further sectors via BIOS disk services) — the bootloader cannot simply be larger than one sector and expect BIOS to load more of it automatically.

Why is `0x7C00` significant?

`0x7C00` is the fixed physical memory address at which BIOS loads the boot sector before jumping to it. This address became a de facto standard on IBM PC-compatible BIOS implementations early on — chosen to sit above the memory BIOS itself was using at boot time (e.g. the interrupt vector table and BIOS data area at the very bottom of memory) while leaving room below the conventional memory ceiling — and every BIOS since has continued to honor it for compatibility.

What does the `0x55AA` boot signature represent?

It is a 2-byte magic value (bytes `0x55`, `0xAA`, stored little-endian as the word `0xAA55`) that BIOS checks at offset 510–511 of the first sector of a bootable device. If those two bytes are not present, BIOS considers the device non-bootable and will not transfer control to that sector's

code, even if the code preceding it is otherwise valid — it is BIOS's minimal sanity check that the sector was intentionally prepared as a boot sector.

How is QEMU involved in the boot process?

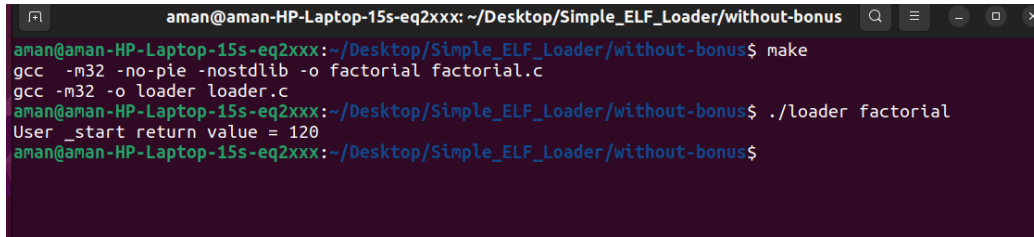
QEMU emulates a complete PC platform — CPU, BIOS (SeaBIOS in this setup), and disk controller — in software. Pointing QEMU's virtual disk at boot.bin makes it behave exactly as real PC firmware would with a physical bootable disk: SeaBIOS loads the first 512 bytes of the image to 0x7C00, verifies the 0x55AA signature, and jumps to it. This allows the bootloader to be developed and tested without needing physical hardware, a real disk, or the risk of overwriting an actual boot sector.

5. Testing & Verification

Each build was verified by actually compiling and running it end-to-end, not just inspected for correctness. The screenshots below are taken directly from the terminal/QEMU sessions used to validate this submission.

5.1 Part A — without-bonus

factorial(5) is computed entirely inside the loaded ELF segment and its return value is retrieved via the function-pointer call to `_start`:

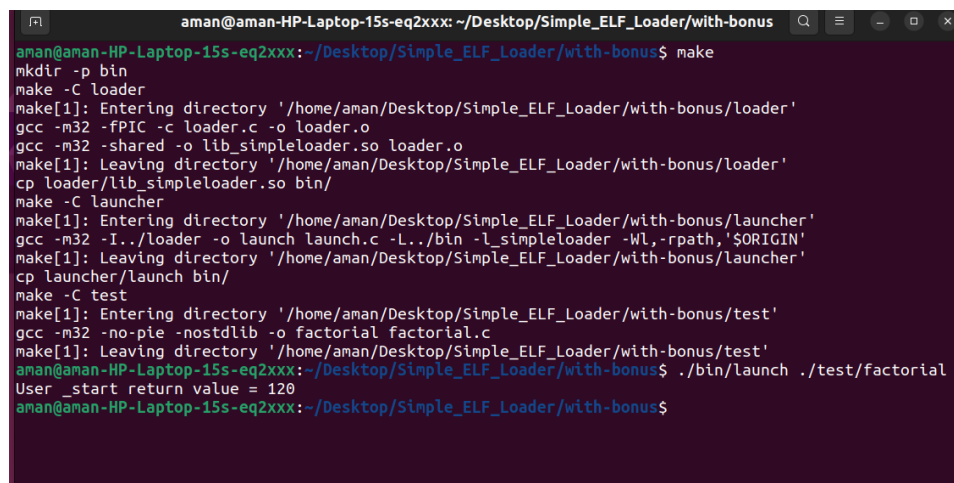


```
aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Simple_ELF_Loader/without-bonus
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/without-bonus$ make
gcc -m32 -no-pie -nostdlib -o factorial factorial.c
gcc -m32 -o loader loader.c
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/without-bonus$ ./loader factorial
User _start return value = 120
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/without-bonus$
```

make → ./loader factorial — output: User _start return value = 120

5.2 Part A — with-bonus

Full build log showing the loader/, launcher/, and test/ sub-Makefiles each being invoked in turn by the top-level Makefile, followed by the launcher successfully loading the shared library and executing the test case:



```
aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Simple_ELF_Loader/with-bonus
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/with-bonus$ make
mkdir -p bin
make -C loader
make[1]: Entering directory '/home/aman/Desktop/Simple_ELF_Loader/with-bonus/loader'
gcc -m32 -fPIC -c loader.c -o loader.o
gcc -m32 -shared -o lib_simpleloader.so loader.o
make[1]: Leaving directory '/home/aman/Desktop/Simple_ELF_Loader/with-bonus/loader'
cp loader/lib_simpleloader.so bin/
make -C launcher
make[1]: Entering directory '/home/aman/Desktop/Simple_ELF_Loader/with-bonus/launcher'
gcc -m32 -I../loader -o launch launch.c -L../bin -l_simpleloader -Wl,-rpath,'$ORIGIN'
make[1]: Leaving directory '/home/aman/Desktop/Simple_ELF_Loader/with-bonus/launcher'
cp launcher/launch bin/
make -C test
make[1]: Entering directory '/home/aman/Desktop/Simple_ELF_Loader/with-bonus/test'
gcc -m32 -no-pie -nostdlib -o factorial factorial.c
make[1]: Leaving directory '/home/aman/Desktop/Simple_ELF_Loader/with-bonus/test'
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/with-bonus$ ./bin/launch ./test/factorial
User _start return value = 120
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/with-bonus$
```

make → ./bin/launch ./test/factorial — output: User _start return value = 120

5.3 Part B — Bootloader

QEMU booting boot.bin from a virtual disk, with SeaBIOS handing control to the bootloader at 0x7C00:

Assembling and launching under QEMU:


```
aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Simple_ELF_Loader/Complete_bootloader
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/with-bonus$ cd ..
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader$ cd Complete_bootloader
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/Complete_bootloader$ nasm -f bin complete_
bootloader.asm -o boot.bin
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Simple_ELF_Loader/Complete_bootloader$ qemu-system-x86_64 -d
rive format=raw,file=boot.bin
```

nasm -f bin complete_bootloader.asm -o boot.bin && qemu-system-x86_64 -drive format=raw,file=boot.bin

```
QEMU
Machine View
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8B590+07ECB590 CA00

Booting from Hard Disk...
Hello from my bootloader!_
```

QEMU window: SeaBIOS hands control to the boot sector at 0x7C00, which prints the message and halts

6. Key Learnings

- The distinction between a file's virtual addresses (`p_vaddr`, `e_entry`) and where it actually ends up in memory at runtime — and that the loader's job is precisely to bridge that gap.
- Why static linking and `-no-pie` matter: they are what make it valid to treat ELF-recorded addresses as fixed offsets, without needing a relocation or symbol-resolution step.
- How a shared library and its consumer only need to agree on a header's declared interface — the launcher never needs to see `loader.c`'s internals, only `loader.h`.
- How little code actually executes before an operating system exists at all: the entire bootloader — enough to print text to the screen and safely halt — fits inside 512 bytes.
- The practical importance of cross-checking an implementation against the exact figures/constraints an assignment specifies (e.g. `p_filesz`), rather than only against general documentation for the format being implemented.

7. Repository

Full source code, Makefiles, and build/run instructions:

<https://github.com/aman24063/CSE231-Programming-Assignments-2026>

Assignment 1 is located under the Simple_ELF_Loader/ directory of the repository, containing without-bonus/, with-bonus/, and Complete_bootloader/ subfolders as described in Sections 2 and 4.

8. Conclusion

This assignment implements two independent, minimal pieces of the program-execution stack: a userspace ELF loader that manually replicates the segment-mapping and entry-point-transfer steps an OS loader performs during `exec()`, and a boot sector that demonstrates how a machine transitions from BIOS firmware to running arbitrary code with no operating system present at all. Both were tested and verified to produce the expected output ($\text{factorial}(5) = 120$, and the bootloader's printed message respectively).